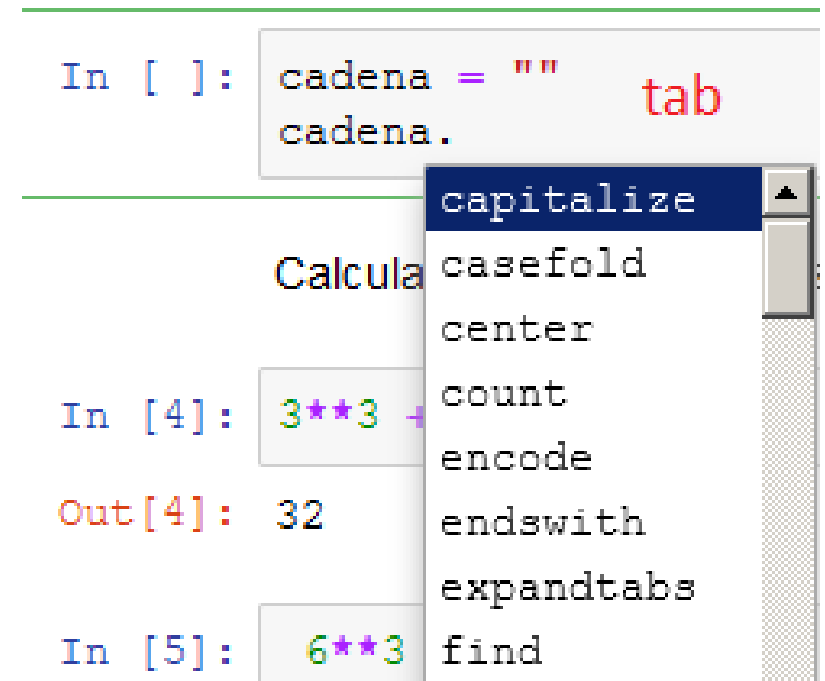


Repàs de Mètodes o funcions de les cadenes

Quan tenim una variable de tipus **cadena**, tenim tot un seguit de funcions o mètodes que podem usar.

Els podem veure al Jupyter Notebook si creen una variable, i després posem el nom de la variable seguida d'un punt i prement la tecla **tabulador**.

A Thonny la tecla és Ctrl + espai.



The screenshot shows a code editor with the following content:

```
In [ ]: cadena = ""  
cadena.
```

A dropdown menu is open, showing the following methods:

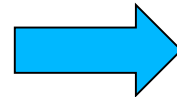
- capitalize
- casefold
- center
- count
- encode
- endswith
- expandtabs
- find

Below the code editor, there is a section for calculations:

```
In [4]: 3**3  
Out[4]: 32  
In [5]: 6**3
```

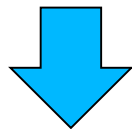
```
s = "Hola mundo"
```

```
print ( s.count ("Hola") )  
print ( s.find("mundo") )  
print ( s.index("mundo") )
```



1
5
5

```
print ( s.find("mondo") )  
print ( s.index("mondo") )
```



-1

```
ValueError                                Traceback (most recent call last)  
<ipython-input-14-e0e79cb09f21> in <module>  
      3 print ( s.count ("Hola") )  
      4 print ( s.find("mondo") )  
----> 5 print ( s.index("mondo") )  
  
ValueError: substring not found
```

En ambos métodos la búsqueda ocurre de izquierda a derecha. Para buscar un conjunto de caracteres desde el final, utilícese del mismo modo `rfind()` y `rindex()`.

```
s = "C:/python36/python.exe"
s.find("/") # Retorna la primera ocurrencia.
2
s.rfind("/") # Retorna la última.
11
```

`startswith()` y `endswith()` indican si la cadena en cuestión comienza o termina con el conjunto de caracteres pasados como argumento, y retornan `True` o `False` en función de ello.

```
s = "Hola mundo"
s.startswith("Hola")
True
s.endswith("mundo")
True
s.endswith("world")
False
```

Canviar lletres o fragments

Tenim el mètode `replace` ..

Sintaxi

```
string.replace(oldvalue, newvalue, count)
```

```
txt = "I like bananas"  
x = txt.replace("bananas", "apples")  
print(x)
```

Valors dels paràmetres

I like apples

Parameter	Description
<i>oldvalue</i>	Required. The string to search for
<i>newvalue</i>	Required. The string to replace the old value with
<i>count</i>	Optional. A number specifying how many occurrences of the old value you want to replace. Default is all occurrences

Comptar elements en una cadena

Retorna el número que cops que troba un valor en una cadena. Es pot especificar la posició inicial i final a cercar dins de la cadena.

```
txt = "I love apples, give me an apple"  
x = txt.count("apple")  
print(x)
```

Syntax

```
string.count(value, start, end)
```

Parameter Values

Parameter	Description
<i>value</i>	Required. A String. The string to value to search for
<i>start</i>	Optional. An Integer. The position to start the search. Default is 0
<i>end</i>	Optional. An Integer. The position to end the search. Default is the end of the string

Exercici P03.1. tweet

```
text = '''EL FOCO\n @elfocoes \n 5 de des. de 2020 \n  
Natalia Lacunza en #Los40MusicAwards2020: \n  
" Nunca colaboraría con Kidd Keo, me parece bastante  
gilipollas. No me escondó " \n @natalialacunza'''
```

#1.- Passa tot el text a minúscules

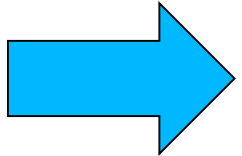
#2.- Canvia els punts i les comes per blancs

#3.- Mira quans cops el text conté la paraula = "gilipollas"

#4.- Mira quans cops el text conté la paraula = "natalia"

#5.- Si fas un find de "saxo", ¿ què retorna ?

#6.- I si fas un find de "de ", ¿ què retorna ?



Si et cal, consulta les funcions replace, count a :

https://www.w3schools.com/python/python_ref_string.a

Los métodos `isdigit()`, `isnumeric()` e `isdecimal()` determinan si todos los caracteres de la cadena son dígitos, números o números decimales.

```
1. >>> "1234".isnumeric()
2. True
3. >>> "1234".isdecimal()
4. True
5. >>> "abc123".isdigit()
6. False
```

Determina si todos los caracteres son alfanuméricos.

```
>>> "abc123".isalnum()
```

```
True
```

Determina si todos los caracteres son alfabéticos.

```
>>> "abcdef".isalpha()
```

```
True
```

```
>>> "abc123".isalpha()
```

```
False
```

Determina si todas las letras son minúsculas.

```
>>> "abcdef".islower()
```

```
True
```

Mayúsculas.

```
>>> "ABCDEF".isupper()
```

```
True
```

```
# Determina si la cadena contiene todos caracteres imprimibles.  
>>> "Hola \t mundo!".isprintable()  
False  
# Determina si la cadena contiene solo espacios.  
>>> "Hola mundo".isspace()  
False  
>>> "    ".isspace()  
True
```

Pràctica P03.2.

#1.- Demana un nom per pantalla

#2.- Si està en majúscula, passa'l a minúscula i viceversa.

#3.- Demana un número enter per pantalla

#4.- Abans de convertir el valor entrat a enter assegurat que la funció int no fallarà

Pràctica P03.3 : Juan Palomo

- 1.- Crea un programa que demani una cadena i la converteixi a majúscules i sense accents. Mostra el resultat.
- 2.- Ara comprova(*) que les 4 cadenes següents, són iguals després de ser transformades :

Juan Palomo Callejón
JUAN PALOMO CALLEJÓN
JUAN PALOMO CALLEJON
Juan palomo callejòn

() Pots comprovar-ho visualment mostrant-les les 4 una a sota de l'altre,
O bé lògicament fent una condició que resulti Veritat si les 4 cadenes són
iguals i Fals si alguna cadena és diferent.*

Cercar dins d' una cadena

Si tenim una variable que es digui paraula i cerquem una lletra (que també tenim en una variable)

```
if lletra in paraula ...      # si troba la lletra
if lletra not in paraula ...  # si no troba la lletra
```

```
# -- resposta menys Pytònica
```

```
if paraula.find(lletra) == -1 ... # si no troba la lletra
if paraula.find(lletra) != -1 ... # si troba la lletra
```

Exemple:

```
if ("sac" in "anem a sac") :
    print ("No tant")
```

breus anotacions

Estructura dels condicionals

```
if a > 1 :           # Si es compleix la condició :
    a = a + 1        # fes tal
else :              # Sinó :
    a = a -1         # fes altre cosa
```

Exemple:

```
vocals = "aeiouàèòáéíóúü"

lletra = input("lletra : ")

if lletra in vocals :
    print ("És una vocal")
else :
    print ("No és una vocal")
```

Exemple:

```
correu = input("e-mail : ")

if "@" not in correu :
    print ("Error: e-mail invàlid, \
          ha de contenir '@' ")
```

Los métodos `center()`, `ljust()` y `rjust()` alinean una cadena en el centro, la izquierda o la derecha respectivamente. Toman un argumento, la cantidad de caracteres respecto de la cual se producirá la alineación.

```
"Hola".center(10)
```

```
'  Hola  '
```

```
"Hola".ljust(10)
```

```
'Hola      '
```

```
"Hola".rjust(10)
```

```
'      Hola'
```

Estos métodos son especialmente útiles al imprimir en forma de tabla para que ésta se mantenga alineada. Un segundo argumento indica con qué caracter se deben llenar los espacios vacíos (por defecto un espacio en blanco).

```
· "Hola".center(10, "*")  
  '***Hola***'
```

Las funciones `strip()`, `lstrip()` y `rstrip()` remueven los espacios en blanco que preceden y/o suceden a la cadena.

```
s = "  Hola mundo!  "
s.strip()
'Hola mundo!'
# Remueve los de la derecha.
s.rstrip()
'  Hola mundo!'
# Remueve los de la izquierda.
s.lstrip()
'Hola mundo!  '
```

Métodos de separación y unión

El método de división de una cadena según un caracter separador más empleado es `split()`, cuyo separador por defecto son espacios en blanco y saltos de línea.

```
"Hola mundo!\nHello world!".split()  
['Hola', 'mundo!', 'Hello', 'world!']
```

El separador puede indicarse como argumento.

```
"Hola mundo!\nHello world!".split(" ")  
['Hola', 'mundo!\nHello', 'world!']
```

O bien, para separar únicamente según saltos de línea:

```
# Equivalente a split("\n").  
"Hola mundo!\nHello world!".splitlines()  
['Hola mundo!', 'Hello world!']
```

Un segundo argumento en `split()` indica cuál es el máximo de divisiones que puede tener lugar (`-1` por defecto para representar una cantidad ilimitada).

```
1. >>> "Hola mundo hello world".split(" ", 2)
2. ['Hola', 'mundo', 'hello world']
```

Por último, el método `join()` —sumamente útil—, que debe ser llamado desde una cadena que actúa como separador para unir dentro de una misma cadena resultante los elementos de una lista.

```
1. >>> " ".join(["Hola", "mundo"])
2. 'Hola mundo'
3. >>> ", ".join(["C", "C++", "Python", "Java"])
4. 'C, C++, Python, Java'
```

Como puede observarse, `split()` y `join()` son exactamente contrarios.

```
1. >>> sep = " "
2. >>> sep.join("Hola mundo!".split(sep))
3. 'Hola mundo!'
```

Pràctica P03.4. csv

```
frase = "Aquest ; és ; un ; fragment ; de csv ; amb; 8; camps \n"
```

1. Si la frase acaba amb un “\n” elimina’l.
2. Divideix la frase en una llista de `camps` usant `split` i el separador “;”
3. Si `camps[6]` és un número, compara’l amb la longitud del la llista `len(camps)` per comprovar si el que diu la frase és cert.

<https://recursospython.com/guias-y-manuales/30-metodos-de-las-cadenas/>

Propina : Com recórrer una llista o array

```
llista = ["cotxe", "carro", "tractor", "moto", "patinet"]
```

```
for element in llista :  
    print (element)
```

```
cotxe  
carro  
tractor  
moto  
patinet
```

P03.5. Avançat : Imprimeix el següent text csv en forma de taula llegible i ben alineada. Canviant el “;” per alguna cosa interessant. I, si vols, fes Split i tracta-ho amb més detall.

```
csv = 'any;mes;dia;obser;ventkh;component;plujamm;tmax;tmin;tmitjana \n \  
1973;12;9;"TARRACO";191;"NO";10;21.22;-2.45;11.83 \n \  
2011;10;24;"BCN";138;"N";6;39.77;16.68;11.55 \n \  
1998;6;6;"VIC";192;"NE";0;19.26;15.06;2.1 \n \  
1987;1;12;"MASNOU";36;"NE";10;29.64;13.4;8.12 \n'
```

SOLUCIONS a P03.5

```
csv = csv.replace(";", "\t")
csv = csv.replace('"', "")
print(csv)
```

anyy	mes	dia	obser	ventkh	component	plujamm	tmax	tmin	tmitjana
1973	12	9	TARRACO	191	NO	10	21.22	-2.45	11.83
2011	10	24	BCN	138	N	6	39.77	16.68	11.55
1998	6	6	VIC	192	NE	0	19.26	15.06	2.1
1987	1	12	MASNOU	36	NE	10	29.64	13.4	8.12

```
lineas = csv.split()
for linea in lineas :
    cad = linea.replace(";", "\t")
    cad = cad.replace('"', "\t")
    print(cad)
```

anyy	mes	dia	obser	ventkh	comp	plujamm	tmax	tmin	tmitjana
1973	12	9	TARRACO	191	NO	10	21.22	-2.45	11.83
2011	10	24	BCN	138	N	6	39.77	16.68	11.55
1998	6	6	VIC	192	NE	0	19.26	15.06	2.1
1987	1	12	MASNOU	36	NE	10	29.64	13.4	8.12



Tipus de dades : Llista

Llistes ordenades de variables

2. Llistes i matrius

2.1. Què són i com funcionen

- Pràctiques P04, P05

3. Introduir dades i mostrar per pantalla

3.1. Comanda input()

- Pràctiques P06, P07

3.2. Mostrar dades amb format

3.2.1. Simple : fString

3.2.2. Funció de cadena : format

3.2.3. Avançat: Formateig de xifres

- Pràctiques P08, P09



2.1. Què són i com funcionen

Les llistes en Python són un **tipus de dada** que permet emmagatzemar dades de qualsevol tipus. Són **mutables** i dinàmiques, la qual cosa és la principal diferència amb els sets i les tuples .

Crear llistes Python

Les llistes en Python són un dels tipus o estructures de dades més versàtils de l'llenguatge, ja que permeten emmagatzemar un conjunt arbitrari de dades. És a dir, podem guardar-hi pràcticament el que sigui. Si véns d'altres llenguatges de programació, es podria dir que **són similars als arrays**.

```
llista = [1, 2, 3, 4]
```

en castellano <https://ellibrodepython.com/listas-en-python>

Una llista sigui crea amb [] separant els seus elements amb comes ,.
Un gran avantatge és que poden emmagatzemar tipus de dades diferents.

```
llista = [1, "Hola", 3.67, [1, 2, 3]]
```

Algunes propietats de les llistes:

- Són ordenades , mantenen l'ordre en què han estat definides
- Poden ser formades per tipus arbitraris
- Poden ser indexades amb [i].
- Es poden niar , és a dir, ficar una dins de l'altra.
- Són mutables , ja que els seus elements poden ser modificats.
- Són dinàmiques , ja que es poden afegir o eliminar elements

```
: llista = []  
print("punto 1:", llista)  
  
llista [0] = "a"
```

punto 1: []

```
-----  
IndexError                                Traceback  
<ipython-input-12-4b536ce77f20> in <module>  
      2 print("punto 1:", llista)  
      3  
----> 4 llista [0] = "a"
```

IndexError: list assignment index out of range

... llavors he d'afegir els elements amb **append**.

Si vull crear una llista inicialment buida i anar-la creant sobre la marxa ...

```
: llista = []  
print("punto 1:", llista)  
  
llista.append("a")  
llista.append("b")  
llista.append("c")  
  
print("punto 2:", llista)
```

punto 1: []
punto 2: ['a', 'b', 'c']

Examples



```
In [1]: numeros = [1,2,3,4]

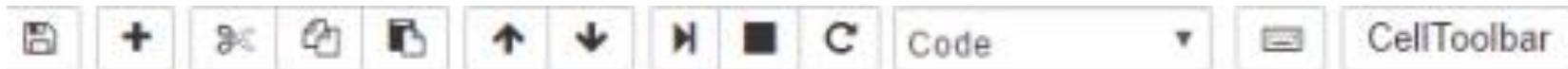
In [2]: datos = [4,"Una cadena",-15,3.14,"Otra cadena"]

In [3]: datos[0]
Out[3]: 4

In [4]: datos[-1]
Out[4]: 'Otra cadena'

In [5]: datos[-2:]
Out[5]: [3.14, 'Otra cadena']

In [6]: numeros + [5,6,7,8]
Out[6]: [1, 2, 3, 4, 5, 6, 7, 8]
```



```
In [7]: pares = [0,2,4,5,8,10]
```

```
In [8]: pares[3]= 6
```

Diferencia amb les cadenes

```
In [9]: pares
```

```
Out[9]: [0, 2, 4, 6, 8, 10]
```

```
In [10]: pares.append(12)
```

```
In [11]: pares
```

```
Out[11]: [0, 2, 4, 6, 8, 10, 12]
```

```
In [12]: pares.append(7*2)
```

```
In [13]: pares
```

```
Out[13]: [0, 2, 4, 6, 8, 10, 12, 14]
```



```
Out[13]: [0, 2, 4, 6, 8, 10, 12, 14]
```

```
In [14]: letras = ['a','b','c','d','e','f']
```

```
In [15]: letras[:3]
```

```
Out[15]: ['a', 'b', 'c']
```

```
In [16]: letras[:3] = ['A','B','C']
```

```
In [17]: letras
```

```
Out[17]: ['A', 'B', 'C', 'd', 'e', 'f']
```

```
In [18]: letras[:3] = []
```

```
In [19]: letras
```

```
Out[19]: ['d', 'e', 'f']
```

```
In [21]: letras = []
```

```
In [22]: letras
```

```
Out[22]: []
```

```
In [23]: len(letras)
```

```
Out[23]: 0
```

```
In [24]: len(pares)
```

```
Out[24]: 8
```

```
In [25]: a = [1,2,3]
         b = [4,5,6]
         c = [7,8,9]
         r = [a,b,c]
```

Es crea una llista de llistes o

```
In [26]: r
```

matriu

```
Out[26]: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
In [27]: r[0]
```

```
Out[27]: [1, 2, 3]
```

```
In [28]: r[-1]
```

```
Out[28]: [7, 8, 9]
```

```
In [29]: r[0][0]
```

```
Out[29]: 1
```

```
In [30]: r[1][1]
```

```
Out[30]: 5
```

```
In [31]: r[2][2]
```

```
Out[31]: 9
```

```
In [32]: r[-1][-1]
```

```
Out[32]: 9
```

També es pot crear fent servir `list` i *passant un objecte iterable*

```
llista = list("1234")
```

```
llista = list("1234")  
print(llista)
```

```
['1', '2', '3', '4']
```

```
llista1 = list(" " * 3)  
print("punt 1:", llista1)
```

```
llista2 = list("XOX")  
print("punt 2:", llista2)
```

```
llista3 = list(" X ")  
print("punt 3:", llista3)
```

```
punt 1: [' ', ' ', ' ']
```

```
punt 2: ['X', 'O', 'X']
```

```
punt 3: [' ', 'X', ' ']
```

P03.6. Pràctica de llistes unidimensionals:

```
# Tenint la variable llista = [a,e,i,o,u], imprimeix  
# llista[1:4]  
# llista[:5]  
# llista[-4:]  
# llista[2]  
# llista[:]  
# llista[::-1]
```

Avançat P03.7. Llistes bi-dimensionals:

```
# Tenim aquest joc de 3 en ratlla.  
# J |   | O  
#   | J | O  
#   |   |  
# 1.Crea una taula bidimensional (t).  
# 2.Posa les marques i fitxes inicials.  
# 3.Imprimeix el taulell (t)  
# 4.Demana una jugada de Jugador J(linia, columna)  
# 5.Coloca la jugada al taulell (t)  
# 6.Torna a imprimir el taulell (t)
```

J		O
	J	O

P03.8. Pràctica de demanar dades

Demana un número de mes (calendari)
i mostra el seu nom en lletres.
hauries de tenir una llista amb els noms ["gener", "febrer"..etc]

Pista :

Pensa que el mes que has demanat ha de servir d'index de la llista de mesos, per tant hauràs de convertir el valor demanat per assegurar que és un número.

Algunes solucions

```
llista = ['a','e','i','o','u']
```

#P03.6 - calcula

```
print(llista[1:4])
```

```
print(llista[:5])
```

```
print(llista[-4:])
```

```
print(llista[2])
```

```
print(llista[:])
```

```
print(llista[::-1])
```

```
['e', 'i', 'o']
```

```
['a', 'e', 'i', 'o', 'u']
```

```
['e', 'i', 'o', 'u']
```

```
i
```

```
['a', 'e', 'i', 'o', 'u']
```

```
['u', 'o', 'i', 'e', 'a']
```

SOLUCIÓ P03.7

```
# Tenim aquest joc de 3 en ratlla.  
# J |   | O  
#   | J | O  
#   |   |  
# Crea una taula bidimensional (t). Posa la situació inicial.  
# Imprimeix el taulell (t)  
# Demana una jugada de Jugador J(linia, columna)  
# Coloca la jugada al taulell (t)  
# torna a imprimir el taulell (t)
```

```
t = [list ("J O"), list (" JO"), list ("  ")]
```

```
# ----- imprime tablero  
print (t[0], t[1], t[2], sep="\n")
```

```
lin = int(input("Linea (1-3): "))  
col = int(input("Columna (1-3): "))  
t[lin -1] [col -1] = "J"
```

```
# ----- imprime tablero  
print (t[0], t[1], t[2], sep="\n")
```

Si creem aquesta funció i la fem servir, quedarà molt millor la impressió del taulell.

```
def printau (f1, f2, f3) :  
    print (f1[0], "|", f1[1], "|", f1[2])  
    print (f2[0], "|", f2[1], "|", f2[2])  
    print (f3[0], "|", f3[1], "|", f3[2])
```